

Disciplina Técnicas de Programação 1

Profa. Roberta Barbosa Oliveira

Sistema de Gestão de Copa do Mundo

Grupo 9 – 2026.1

Danilo Lins Castro – 251019735 – 251019735@aluno.unb.br
Helton Rodrigues da Silva Júnior – 251021789 – 251021789@aluno.unb.br
Ighor Gomes das Chagas – 221020913 – 221020913@aluno.unb.br
João Pedro Barros Silva Azevedo – 251004311 – 251004311@aluno.unb.br
João Vítor da Silva Lima – 251023101 – 251023101@aluno.unb.br

Resumo

O presente trabalho expõe os detalhes sobre o processo de desenvolvimento de um sistema de Gestão para a Copa do Mundo de 2026, voltado para uso Desktop. São descritas todas as etapas da criação do software - análise de domínio, design e implementação; além de uma investigação retrospectiva da produção da aplicação.

1 Introdução

1.1 Contexto do Projeto

Neste projeto, busca-se desenvolver um sistema de gestão da Copa do Mundo de futebol. Assim, espera-se tratar de aspectos naturais do universo desse esporte, como partidas, seleções, jogadores, árbitros e público, todos mutuamente dependentes e essenciais para o bom funcionamento da plataforma. Para isso, o grupo lançará mão de conhecimentos obtidos ao longo da disciplina de Técnicas de Programação 1, do Departamento de Ciência da Computação, de modo a abordar, principalmente, tópicos relacionados a Programação Orientada a Objetos.

1.2 Objetivos de Desenvolvimento

De maneira geral, o grupo espera desenvolver um sistema coerente, coeso e que cumpre o problema proposto com excelência. Assim, valorizam-se, especialmente, a modularização, a

qualidade de software e a robustez do projeto, fatores alcançados por meio da Programação Orientada a Objetos e tudo que a circunda, como polimorfismo, encapsulamento, heranças, tratamento de exceções e padrões de desenvolvimento. Ademais, os alunos almejam, além do desenvolvimento individual, aperfeiçoar-se no trabalho em equipe, com foco na produção conjunta e no uso de ferramentas colaborativas dominantes no mercado.

1.3 Requisitos Funcionais

1.3.1 Administração/Gestão

- Cadastrar, editar, excluir e listar contas de usuários do sistema (administradores, organizadores, árbitros, operadores)
- Restringir funcionalidades de acordo com o perfil de acesso.
- Permitir pesquisa de usuários por múltiplos critérios (nome, função, país, status).
- Validar dados de usuários (e-mail, identificação).
- Gerar relatórios consolidados da competição (número de partidas, público, desempenho de seleções).

1.3.2 Seleções e Jogadores

- Cadastrar, editar, excluir e listar seleções (país, grupo, técnico).
- Cadastrar, editar, excluir e listar jogadores (nome, posição, número, idade, seleção).
- Associar jogadores a uma seleção.
- Controlar status dos jogadores (ativo, lesionado, suspenso).
- Consultar seleções e jogadores por critérios (grupo, posição, status).

1.3.3 Estádios e Arbitragem

- Cadastrar, editar, excluir e listar estádios (nome, localização, capacidade).
- Associar partidas a estádios.
- Cadastrar árbitros (nome, nacionalidade, experiência).
- Designar árbitros para partidas.
- Consultar estádios e árbitros por critérios.

1.3.4 Partidas

- Cadastrar, editar, excluir e listar partidas (data, horário, estádio, seleções).
- Definir fase da competição (grupos, oitavas, quartas, semifinal, final).
- Registrar resultados das partidas (placar, eventos).
- Atualizar status das partidas (agendada, em andamento, finalizada).
- Consultar partidas por seleção, fase ou data.

1.3.5 Ingressos e Público

- Gerenciar venda de ingressos por partida.
- Definir categorias de ingressos (VIP, padrão, meia-entrada).
- Controlar quantidade disponível por partida.
- Registrar compra de ingressos.
- Gerar relatórios de público e arrecadação.

1.4 Requisitos não Funcionais

- Ter interface amigável para interação (por exemplo: Swing, JavaFX ou menu de opções em prompt).
- Oferecer usabilidade adequada (botões e menus claros, feedback ao usuário, alertas antes de exclusão definitiva).
- Exibir mensagens claras para erros de regra de negócio, usando exceções personalizadas.
- Deve ser implementado de forma orientada a objetos, garantindo modularidade e permitindo a extensão de funcionalidades sem impactar de forma significativa as classes já existentes.
- A modelagem deve priorizar herança e composição para reaproveitamento de código e redução de duplicidade.
- Deve respeitar princípios de encapsulamento, expondo apenas os métodos necessários ao uso externo.
- Deve adotar boas práticas de POO, como responsabilidade única (altamente coesa) e baixo acoplamento entre módulo, para facilitar manutenção e evolução do sistema.
- Empregar pelo menos dois padrões de projeto (por exemplo: Singleton, Factory, Builder).
- Ser implementado em camadas bem definidas (por exemplo: apresentação, aplicação, domínio, persistência, infraestrutura).
- Armazenar dados em arquivos (por exemplo: texto, binário ou serialização de objetos).
- Ser executável em qualquer sistema operacional com Java instalado.
- Garantir autenticação obrigatória via login e senha.
- (Opcional) Suporte a concorrência, por exemplo para venda de ingressos.

1.5 Regras de negócio

1.5.1 Administração/Gestão

- Perfis de acesso são hierárquicos: administrador (acesso total), organizador (gestão de partidas e seleções), árbitro (visualização de partidas designadas), operador (ingressos e registros).
- Administradores podem acessar todas as categorias, enquanto organizador, árbitro e operador têm acesso restrito conforme perfil.
- Apenas administradores podem criar, editar ou remover contas de outros usuários.
- Senhas devem respeitar políticas mínimas (por exemplo: 8 caracteres, letras e números).

1.5.2 Seleções e Jogadores

- Cada jogador deve estar vinculado a uma única seleção.
- Uma seleção deve possuir número mínimo e máximo de jogadores (exemplo: 18 a 26).
- Jogadores suspensos ou lesionados não podem participar de partidas.

1.5.3 Estádio e Arbitragem

- Um estádio não pode sediar duas partidas no mesmo horário.
- Cada partida deve ter ao menos um árbitro principal.
- Árbitros não podem atuar em partidas de seleções de sua nacionalidade.

1.5.4 Partidas

- Uma partida deve envolver exatamente duas seleções diferentes.
- Uma seleção não pode jogar duas partidas no mesmo horário.
- Partidas devem ser associadas a um estádio.
- Apenas partidas finalizadas podem ter resultados registrados definitivamente.

1.5.5 Ingressos e Público

- A quantidade de ingressos não pode exceder a capacidade do estádio.
- Ingressos não podem ser vendidos após o início da partida.
- Cada ingresso deve estar vinculado a uma partida específica.
- Relatórios financeiros devem considerar apenas ingressos vendidos.

1.5.6 Atores

- **Administrador:** Responsável pelo controle geral do sistema, usuários e configurações.
- **Organizador:** Gerencia seleções, jogadores, partidas e estádios.
- **Árbitro:** Visualiza partidas designadas e informações relacionadas.
- **Operador:** Responsável pela venda de ingressos e controle de público.
- **Espectador:** Não acessa diretamente o sistema (representado no controle de ingressos).

1.6 Estrutura do Relatório

As próximas seções do relatório detalham a análise e o desenvolvimento do software implementado. Na Seção 2 são apresentadas a modelagem, a estrutura, os módulos, a arquitetura e as funcionalidades desenvolvidas, descrevendo como cada etapa do projeto contribuiu para a construção do software. Na Seção 3, discute-se a análise das funcionalidades desenvolvidas, destacando resultados alcançados, integração entre os módulos, confiabilidade, usabilidade e outras características do software, além de eventuais limitações ou desafios encontrados durante o desenvolvimento. Por fim, a Seção 4 sintetiza a relevância do trabalho desenvolvido, avaliando seu impacto no contexto aplicado e apontando possíveis melhorias e extensões para evoluções futuras.

Nos relatórios parciais, devem ser incluídas apenas as partes que já foram desenvolvidas até o momento. Cada entrega deve corresponder a uma etapa específica da seção de Solução Proposta: Estrutura Inicial, Design e Modelagem, Implementação da Lógica do Software e Integração e Navegabilidade. Nessas versões, é obrigatório incluir o Resumo e a Introdução, enquanto a Solução Proposta deve refletir a etapa de desenvolvimento atual, detalhando funcionalidades, diagramas e técnicas de programação aplicadas. Alterações ou melhorias em relação a versões anteriores devem ser documentadas na versão atual.

No relatório final, o documento deve ser revisado e atualizado, mantendo apenas o conteúdo referente ao que foi de fato desenvolvido, contemplando todas as implementações, funcionalidades, diagramas, módulos, técnicas de programação aplicadas e integração final. Além do Resumo, da Introdução e da Solução Proposta, o relatório final deve incluir as seções Resultados e Discussão, e Conclusão e Evolução Futura, para refletir a solução completa desenvolvida pelo grupo e sua relevância para o domínio aplicado.

2 Solução Proposta

2.1 Estrutura Inicial

Na definição das ferramentas, decidiu-se pelo Git e GitHub para controle de versionamento do projeto; NetBeans para Interface de Desenvolvimento Integrado (IDE), Maven para gestão e automação de dependências; e o Swing como biblioteca gráfica, em virtude da sua integração de design com a IDE. O repositório do projeto no GitHub está disponível em: <https://github.com/vitor2828/tp1>

A estruturação em pacotes foi definida conforme a seguinte disposição:

```
sistema.gestao.copa.do.mundo/  
  src/  
    administracao.gestao/  
      telas  
      classes  
      excecoes  
    selecao.jogador/  
      telas  
      classes  
      persistencia  
    estadio.arbitragem/  
      telas  
      classes  
    partidas/  
      telas  
      classes  
    ingressos.publico/  
      gui  
      classes  
  relatorios  
  pom.xml  
  README.md
```

O desenvolvimento foi repartido entre os membros constituintes segundo a distribuição abaixo.

- **João Vitor - Administração e Gestão:** gestão de usuários, autenticação, controle de acesso e relatórios gerais. Classes: Usuario, Administrador, Organizador, Operador. Telas: Login, Gestão de Usuários, Relatórios, Tela Inicial
- **João Pedro - Seleções e Jogadores:** gestão de seleções e elenco de jogadores. Classes: Selecao, Jogador. Telas: Cadastro de Seleções, Cadastro de Jogadores, Consulta
- **Helton Rodrigues - Estádios e Arbitragem:** gestão de estádios e árbitros. Classes: Estadio, Arbitro, DesignacaoArbitro. Telas: Cadastro de Estádios, Cadastro de Árbitros, Designação Árbitro
- **Danilo Lins - Partidas:** controle das partidas e resultados. Classes: Partida, Fase, Resultado. Telas: Cadastro de Partidas, Registro de Resultados, Consulta de Jogos.
- **Ighor Gomes - Ingressos e Público:** controle de ingressos e arrecadação. Classes: Ingresso, Venda, CategoriaIngresso. Telas: Venda de Ingressos, Controle de Público, Relatórios Financeiros

2.2.1 Diagrama de Classes

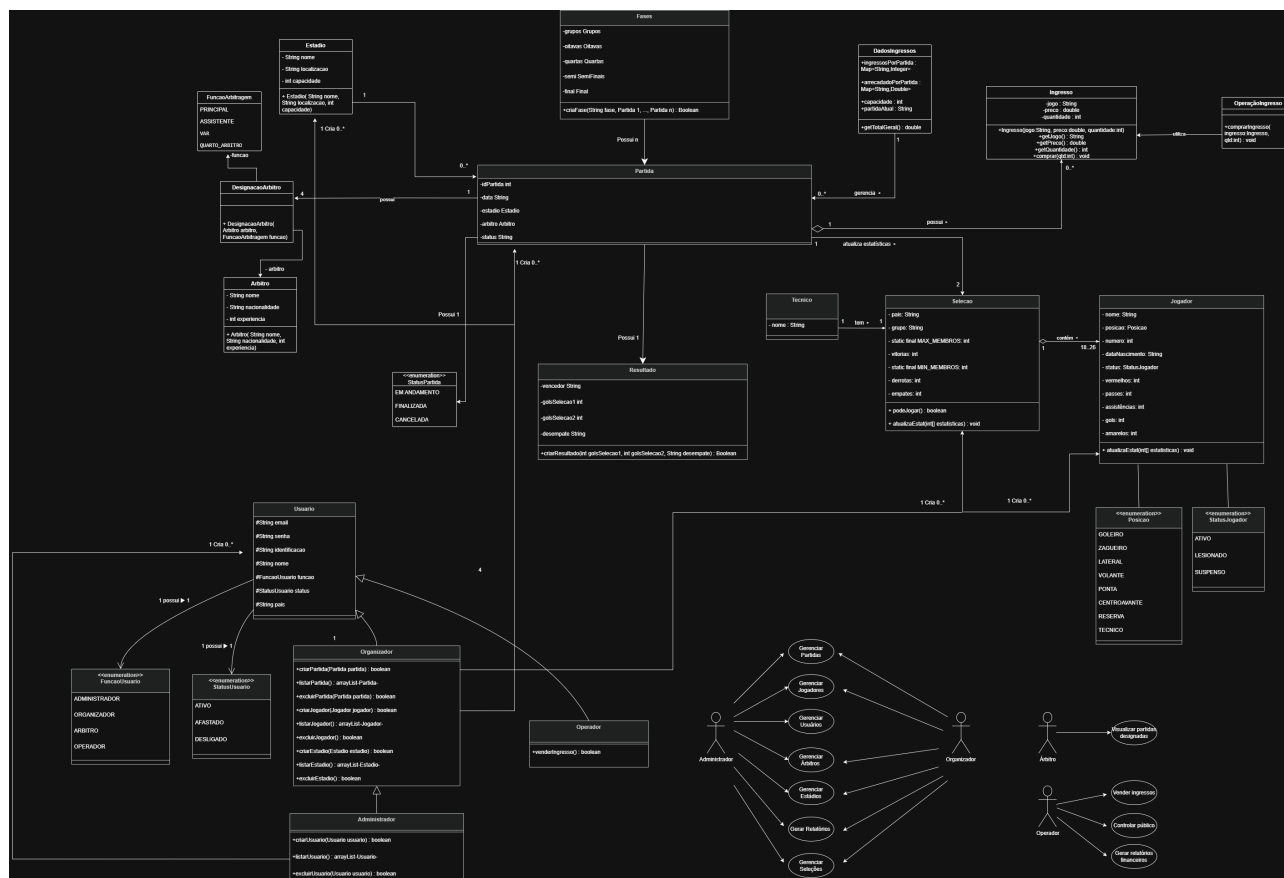


Figura 1: Diagrama de Classes de Domínio

Descrição das Classes

2.2.2 Módulo Administração

- **Classe Usuário:** Essa classe é responsável por reunir todos os atributos comuns aos usuários do sistema. Dessa maneira, há diversas informações básicas, como nome, email, senha, país, identificação e statusUsuario, uma Enum que agrupa todos os valores possíveis para esse atributo, como ATIVO, AFASTADO e DESLIGADO. Por fim, há uma associação à classe Papel, que diz qual o cargo que esse usuário ocupa na plataforma.
- **Classe UsuarioLogado:** Essa classe é utilizada para controlar a sessão atual da plataforma, central para validação de permissões. Com isso, contém apenas um atributo final do tipo Usuario, para que não seja possível modificar informações em outras partes do sistema.

- **Classe Papel:** Uma classe abstrata cujo objetivo principal é agrupar todos os cargos possíveis em uma estrutura só. Assim, não tem muito conteúdo, apenas um método abstrato que retorna o nome do cargo em questão.
- **Classe Permissao:** Da mesma maneira que Papel, é uma classe abstrata que agrupa as mais diversas permissões, as quais validam se um usuário pode efetuar certa ação.
- **Classe Organizador:** É uma especialização direta de Papel, caracterizando um usuário que pode criar/editar/excluir partidas, seleções, jogadores e árbitros. Com isso, seu conjunto de permissões faz jus a esses métodos.
- **Classe Operador:** É uma outra especialização direta de Papel. No entanto, suas permissões permitem o usuário realizar operações financeiras, como a venda de ingressos e a geração de relatórios.
- **Classe Administrador:** Essa classe tem acesso total ao sistema, ou seja, está no topo da hierarquia de usuários. Entretanto, possui a permissão exclusiva de criar/editar/excluir outros usuários. Dessa forma, também é uma especialização de Papel, mas com todas as permissões da plataforma.
- **Classe AdministraUsuario:** Sendo uma especialização de Permissao, AdministraUsuario contém todos os métodos relacionados ao CRUD de usuários. Se um Papel tem essa classe entre suas permissões, o Usuario ao qual pertence poderá realizar essas operações.
- **Classe GeraRelatorios:** Lógica análoga a AdministraUsuario, porém com métodos relacionadas à geração de relatórios administrativos, contendo o número de usuários cadastrados e seus tipos.

2.2.3 Módulo Jogadores e Seleções

- **Classe Jogador:** Essa classe é responsável por armazenar e editar os elementos inerentes a cada jogador. Destacam-se as enumerações Posicao e StatusJogador, usadas para padronizar a categorização de jogadores. Seu único método é um construtor parcial, o qual não recebe os elementos de performance (faltas, cartões vermelhos, amarelos, etc) como parâmetros.
- **Classe Selecao:** Classe encarregada de organizar os elementos de cada Seleção. Apresenta constantes de mínimo e máximo de membros constituintes, para atender às regras de negócio; e agrega o time em uma tabela Hash com tratamento de colisões por Lista Linkada para garantir escalabilidade do sistema de gestão como um todo. Vale ressaltar que o único construtor da classe modifica múltiplos jogadores, enquanto, os demais métodos alteram somente um jogador por chamada.

2.2.4 Estádios e Arbitragem

- **Classe Estadio:** Responsável por representar e armazenar as informações da infraestrutura, como nome, localização e capacidade. Além do método construtor para

inicialização dos objetos, possui métodos de acesso e modificação (Getters e Setters) para garantir o encapsulamento dos atributos privados.

- **Classe Arbitro:** Representa os árbitros cadastrados no sistema, contendo atributos básicos de identificação e qualificação (como nacionalidade e nível de experiência). Assim como a classe Estádio, utiliza métodos Getters e Setters para controle seguro de acesso aos dados, além do seu construtor padrão.
- **Classe DesignacaoArbitro:** Atua como uma associação lógica. Seus únicos atributos são uma instância da classe **Arbitro** e um valor da enumeração **FuncaoArbitragem**, que são os elementos necessários para alocar a equipe de arbitragem para cada partida. O acesso aos seus dados também é protegido por encapsulamento, sendo inicializada através de seu método construtor.

2.2.5 Partidas

- **Classe Partida:** Essa classe tem o objetivo de armazenar os dados que representam cada partida dentro da competição como código, data, onde ocorreu, o árbitro designado e o andamento da partida. Os métodos para o cadastro das informações estão na classe Organizador, como dito anteriormente.
- **Classe Fase:** Responsável por representar as diferentes etapas da competição, como grupos, oitavas, quartas, semifinais e final. A classe utiliza uma enumeração para padronizar as fases disponíveis no sistema e possui um método de validação capaz de criar uma fase e verificar que ela está de acordo com outras etapas já criadas, garantindo consistência na organização da competição.
- **Classe Resultado:** Classe encarregada de armazenar e representar o desfecho de cada partida. Seus atributos contemplam informações como quantidade de gols de cada seleção, existência de desempate e seleção vencedora. Além disso, possui um método responsável por criar e validar os resultados registrados, assegurando coerência entre os dados estatísticos da partida e o resultado final apresentado.

2.2.6 Módulo Ingressos e Público

- **Classe Ingresso:** Representa os ingressos disponíveis para cada partida do sistema, armazenando informações como categoria, valor e demais dados relacionados à compra. Essa classe serve como modelo para o gerenciamento dos ingressos comercializados durante a competição.
- **Classe DadosIngressos:** Responsável por armazenar e gerenciar os dados referentes às vendas de ingressos e ao público das partidas. Mantém informações como quantidade de ingressos vendidos, arrecadação por partida, capacidade do estádio e partida atualmente selecionada. Além disso, realiza o carregamento automático das informações persistidas no arquivo `eventos.json`, permitindo que os dados sejam recuperados quando o sistema é iniciado.

- **Classe GerenciaIngressos:** Implementa as principais regras de negócio do módulo de ingressos. É responsável pelo processamento da compra de ingressos, validação da quantidade permitida por compra, controle da capacidade máxima do estádio, atualização da arrecadação financeira, geração e persistência do relatório financeiro em `eventos.json`, cálculo de indicadores como total arrecadado, média de público e evento de maior arrecadação, além da funcionalidade de reinicialização da temporada, que limpa os dados estatísticos das vendas.
- **Classe OperacaoIngresso:** Responsável por centralizar as operações relacionadas aos ingressos, auxiliando na organização das funcionalidades do módulo e na integração entre a interface gráfica e as regras de negócio implementadas.

2.3 Implementação da Lógica do Software

2.3.1 Administração

Antes de tudo, é importante elucidar dois padrões de design que foram essenciais para arquitetar essa seção: o RBAC (Role-Based Access Control, ou Controle de Acesso Baseado em Papéis) e a Injeção de Dependência. Além disso, deve-se tecer alguns comentários acerca da escolha do modelo de persistência.

O primeiro é utilizado para restringir o acesso ao sistema a depender do tipo de usuário atual. Em geral, é definido por uma classe Papel, que, associada a um Usuario, é verificada para certificar a ação. Isso, na teoria, já é suficiente para estruturar uma lógica de permissões sólida. No entanto, para aumentar a modularidade e a escalabilidade da plataforma, optou-se por, em cada Papel, dispor de uma coleção de permissões, representadas pela classe Permissao. Isso permite que cada cargo possa ter suas permissões facilmente verificadas e modificadas sem alterar outras partes do código, de modo a seguir boas práticas de desenvolvimento de software, destacando-se o SOLID.

O segundo, por sua vez, é responsável por obter informações do usuarioLogado, de forma a alimentar o RBAC. Sob uma primeira análise, uma maneira simples de fazer isso seria transformar usuarioLogado em uma classe Singleton, isto é, instanciada apenas uma vez e com os atributos estáticos, o chamado Padrão Singleton. No entanto, essa lógica é vista por muitos como um anti-padrão de design, de forma a ser evitada em sistemas de larga escala por razões de segurança. Dessa maneira, considera-se a Injeção de Dependência o padrão ideal que substitui o Singleton: trata-se de, toda vez que uma parte do código exigir informações da sessão, passar UsuarioLogado como argumento da função. Assim, o objeto estará restrito ao escopo da função onde é chamado, preservando-o de possíveis problemas.

Por fim, quanto à persistência, o grupo escolheu o JSON (JavaScript Object Notation) com a biblioteca Jackson. Embora o JSON seja, obviamente, nativo do JavaScript, o uso da Jackson possibilitou a conversão fácil entre arquivos de texto e objetos. Isso, comparado com o uso de texto bruto, tornou o trabalho muito mais eficiente, visto que não houve necessidade de criar funções personalizadas para serializar e deserializar os objetos.

Com tudo isso dito, explica-se, agora, a implementação das principais funções referentes a esta seção. Primeiro, destacam-se as funções de AdministraUsuario, que, por padrão, são todas *static*.

```

static public boolean criaUsuario(Usuario usuarioCadastro) throws
    SenhaInsuficienteException {

    if (usuarioCadastro.getSenha().length() < 8) {
        throw new SenhaInsuficienteException("Senha nao cumpre requisitos
            minimos");
    }

    ObjectMapper mapper = new ObjectMapper();
    File persistenciaUsuarios = new File(USUARIOS_FILE_PATH);

    try {
        Map<String, Usuario> mapUsuarios = mapper.readValue(
            persistenciaUsuarios, new TypeReference<Map<String, Usuario>>(){});
        mapUsuarios.put(usuarioCadastro.getIdentificacao(), usuarioCadastro);
        mapper.writeValue(persistenciaUsuarios, mapUsuarios);
    }

    catch (JsonMappingException e) {
        System.err.println("Houve algum problema no mapeamento do JSON durante
            o cadastro do usuario.");
        return false;
    }

    catch (IOException e) {
        System.err.println("Houve algum problema ao manipular o arquivo
            durante o cadastro do usuario.");
        return false;
    }

    return true;
}

```

A função acima é a `criaUsuario()`. Ela recebe uma instancia de `Usuario` e retorna um booleano, que comunica o resultado da operação. Não há muito o que dizer: a função simplesmente inicia o arquivo como um *Map* de `Usuario`, adiciona o novo usuário com seu ID como *key* e reescreve na persistência. Em todas as funções dessa natureza, teremos as exceções `JsonMappingException`, para problemas na análise do JSON, e `IOException`, para problemas na manipulação do arquivo. Por fim, unicamente nesse método, teremos `SenhaInsuficienteException`, uma exceção que, como previsto nas regras de negócio, estipula requisitos mínimos para que a senha seja validada.

```

static public List<Usuario> listaUsuario() {
    ObjectMapper mapper = new ObjectMapper();
    File persistenciaUsuarios = new File(USUARIOS_FILE_PATH);
    List<Usuario> retornaListaUsuarios = new ArrayList<>();

    try {
        Map<String, Usuario> mapUsuarios = mapper.readValue(
            persistenciaUsuarios, new TypeReference<Map<String, Usuario>>(){});

        for (Map.Entry<String, Usuario> entry : mapUsuarios.entrySet()) {
            retornaListaUsuarios.add(entry.getValue());
        }
    }
}

```

```

        catch (JsonMappingException e) {
            System.err.println("Houve algum problema no mapeamento do JSON ao
                               listar usuarios");
        }

        catch (IOException e) {
            System.err.println("Houve algum problema ao manipular o arquivo ao
                               listar usuarios");
        }

        return retornaListaUsuarios;
    }

```

Essa função, por sua vez, é a `listaUsuarios()`. Ela inicia o arquivo JSON como *Map*, itera-o completamente e, a cada loop, adiciona o *Usuario* em questão a uma *List*, a qual será retornada. Essa solução foi escolhida pois é mais fácil integrar listas à interface gráfica do *Swing*.

```

static public List<Usuario> pesquisaUsuario(String nome, String identificacao,
        String email, String pais, String senha, Usuario.StatusUsuario status,
        Papel papel) {
    // TODO: provavelmente nao vai ser elegante criar um poliformismo por
    // overloading.
    // aqui, vou cuidar para, na tela, receber os argumentos. Se ele estiver
    // desabilitado, vou receber como null
    // para ignorar durante a busca.

    ObjectMapper mapper = new ObjectMapper();
    File persistenciaUsuarios = new File(USUARIOS_FILE_PATH);
    List<Usuario> retornaListaUsuarios = new ArrayList<>();

    try {
        Map<String, Usuario> mapUsuarios = mapper.readValue(
            persistenciaUsuarios, new TypeReference<Map<String, Usuario>>(){});

        // 1 caso: se a identificacao nao for nula, ha apenas um retorno.
        // Portanto, pode simplesmente ver se existe no JSON e retornar uma
        // lista unitaria
        // 2 caso: iterar e coletar os usuarios que coincidem com algum
        // parametro.

        if (identificacao.isEmpty() == false && mapUsuarios.containsKey(
            identificacao)) {
            retornaListaUsuarios.add(mapUsuarios.get(identificacao));
        }

        else {
            for (Map.Entry<String, Usuario> entry : mapUsuarios.entrySet()) {
                Usuario usuarioIteracao = entry.getValue();

                if (usuarioIteracao.getNome().equals(nome) || usuarioIteracao.
                    getEmail().equals(email) || usuarioIteracao.getPais().
                    equals(pais)
                    || usuarioIteracao.getStatus() == status ||
                    usuarioIteracao.getPapel().getClass() == papel.getClass()
                    ()) {

```

```

        retornaListaUsuarios.add(usuarioIteracao);
    }

    // isso jah trata a duplicata, porque um usuario vai ser
    // analisado apenas uma vez. Melhor do que fazer um if ou
    // switch case para cada caso
}

}

}

}

catch (JsonMappingException e) {
    System.err.println("Houve algum problema no mapeamento do JSON durante
        a pesquisa de usuarios");
}

catch (IOException e) {
    System.err.println("Houve algum problema ao manipular o arquivo
        durante a pesquisa de usuarios");
}

return retornaListaUsuarios;
}

```

Essa função é a `pesquisaUsuario()`. Da mesma maneira que a `listaUsuario()`, ela estrutura o *Map* da persistência e o percorre. No entanto, somente serão adicionados à *List* de retorno usuários que possuem algum dos parâmetros dados. Em geral, há alguns detalhes dignos de menção: a pesquisa por ID, por retornar apenas um usuário, foi tratada separadamente; não há necessidade de tratar duplicatas, visto que cada usuário do *Map* será iterado uma única vez; com a integração da interface gráfica, valores desconsiderados na busca serão passados como *null*, o que já trará a forma obtida.

```

static public boolean excluiUsuario(Usuario usuario) {
    ObjectMapper mapper = new ObjectMapper();
    File persistenciaUsuarios = new File(USUARIOS_FILE_PATH);

    try {
        Map<String, Usuario> mapUsuarios = mapper.readValue(
            persistenciaUsuarios, new TypeReference<Map<String, Usuario>>() {});
        mapUsuarios.remove(usuario.getIdentificacao());
        mapper.writeValue(persistenciaUsuarios, mapUsuarios);
    }

    catch (JsonMappingException e) {
        System.err.println("Houve algum problema no mapeamento do JSON ao
            excluir usu rio");
        return false;
    }

    catch (IOException e) {
        System.err.println("Houve algum problema ao manipular o arquivo ao
            excluir usu rio");
        return false;
    }

    return true;
}

```

Esse método é o `excluiUsuario()`. Trata-se de uma versão complementar do `criaUsuario`, visto que recebe um usuário como parâmetro, busca-o na persistência e o remove, de modo a, logo depois, atualizar o arquivo.

Agora, quanto à permissão `geraRelatorios`, temos um único método:

```
static public Map<String, Integer> contaUsuarios() {
    int[] vetorAuxiliar = {0, 0, 0, 0, 0};

    ObjectMapper mapper = new ObjectMapper();
    File persistenciaUsuarios = new File(USUARIOS_FILE_PATH);

    try {
        Map<String, Usuario> mapUsuarios = mapper.readValue(
            persistenciaUsuarios, new TypeReference<Map<String, Usuario>>(){});

        for (Map.Entry<String, Usuario> entry : mapUsuarios.entrySet()) {
            vetorAuxiliar[0]++;

            switch (entry.getValue().papel) {
                case Administrador u -> {
                    vetorAuxiliar[1]++;
                }

                case Organizador u -> {
                    vetorAuxiliar[2]++;
                }

                case Operador u -> {
                    vetorAuxiliar[3]++;
                }

                default -> {
                    // nao faz nada
                }
            }
        }
    }

    catch (JsonMappingException e) {
        System.err.println("Houve algum problema no mapeamento do JSON ao
            gerar relat rios de usu rio");
    }

    catch (IOException e) {
        System.err.println("Houve algum problema ao manipular o arquivo ao
            gerar relat rios de usu rio");
    }

    Map<String, Integer> mapaUsuarios = Map.of (
        "numTotalUsuarios", vetorAuxiliar[0],
        "numAdministradores", vetorAuxiliar[1],
        "numOrganizadores", vetorAuxiliar[2],
        "numOperadores", vetorAuxiliar[3],
        "numArbitros", vetorAuxiliar[4]
    );
}
```

```
        return mapaUsuarios;
    }
```

Esse é o `contaUsuarios()`, cuja função é simplesmente iterar a persistência de usuários e contar cada tipo. Retorna um outro *Map*, dessa vez de `String` e `Integer`, respectivamente a chave e o valor, para que os resultados sejam mais fáceis de interpretar e, conseqüentemente, integrar com as telas.

2.3.2 Jogadores e Seleções

Foram exigidas três regras de negócio para este tópico:

1. Cada jogador deve estar vinculado a uma única seleção;
2. Uma seleção deve possuir um número mínimo e máximo de jogadores (exemplo: 18 a 26);
3. Jogadores suspensos ou lesionados não podem participar das partidas.

As regras 1. e 2. são garantidas pela responsabilização exclusiva da classe `Selecao` pela modificação do atributo `selecao` de cada jogador. Mais especificamente, através da adaptação do método `setTime()`.

Para impedir a instanciação de seleções que fujam das regras 1. e 2., o método `setTime()` foi alterado para modificar o atributo `Selecao.time` somente se as condições abaixo forem atendidas. Caso contrário, é lançada uma exceção do tipo `IllegalArgumentException`.

1. Possui quantidade de elementos entre o mínimo e máximo estabelecidos. No caso, entre 18 e 26 incluindo os extremos;
2. Nenhum jogador está previamente vinculado a outra seleção. Isto é, todos os jogadores possuem `selecao = null`.

Quanto a regra 3., ela é regida pela função `podeJogar()`, a qual retorna *true* se não houver jogadores LESIONADOS ou SUSPENSOS no time, retornando *false* caso contrário.

No tocante à persistência de dados, optou-se pela utilização de registros JSON, salvos em arquivos JSONL (*JavaScript Object Notation in Line*), em virtude da escalabilidade e fácil capacidade de inserção (*append*) fornecida por esse modelo de armazenamento. A implementação desta parte, se deu pela biblioteca Jackson, responsável pela captação e escrita de objetos Json, e por um pacote denominado `org.example.persistencia`, o qual agrupa as classes `IOJogador`, `IOSelecao` e `IOTecnico`. Cada uma das classes tem rotinas de inserção, edição, exclusão e análise/leitura dos arquivos .jsonl, direcionadas para acessar, nessa ordem, os arquivos `jogadores.jsonl`, `selecoes.jsonl` e `tecnicos.jsonl`; mantendo ao longo da execução a integridade dos supracitados registros.

Por fim, utilizou-se das exceções `IOException`, `IllegalArgumentException` e `IllegalStateException`; nativas do Java, além da exceção personalizada `ElementoDuplicado`. Cada exceção gera uma janela de diálogo para o usuário avisando que ocorreu um erro na execução e com a descrição deste em texto.

2.3.3 Partidas, Fases e Resultados

Foram definidas regras de negócio específicas para o gerenciamento das partidas e das diferentes fases da competição:

1. Cada partida deve envolver exatamente duas seleções distintas.
2. Apenas seleções classificadas para determinada fase podem participar das partidas dessa fase.
3. Toda partida deve possuir um resultado registrado antes da geração dos classificados para a fase seguinte.
4. Na fase de grupos, a classificação das seleções deve respeitar o sistema de pontuação estabelecido pela competição.
5. Nas fases eliminatórias, cada partida deve obrigatoriamente produzir um vencedor.

A regra 1 é garantida durante a criação das partidas. Antes que uma nova partida seja registrada, o sistema verifica se as duas seleções informadas são diferentes. Caso o usuário tente cadastrar uma partida entre a mesma seleção, uma exceção do tipo `IllegalArgumentException` é lançada, impedindo o registro de dados inconsistentes.

A regra 2 é implementada através da organização das fases da competição. Cada fase possui um arquivo específico contendo as seleções classificadas. Dessa forma, ao criar partidas para uma determinada etapa do torneio, somente as seleções presentes no arquivo de classificados da fase correspondente podem ser utilizadas. Esse mecanismo garante a consistência do chaveamento ao longo da competição.

Quanto à regra 3, ela é garantida pelo método responsável pelo registro de resultados. As seleções somente podem ser promovidas à fase seguinte após todas as partidas da etapa atual possuírem placares devidamente cadastrados. Assim, evita-se a geração prematura de classificados e possíveis inconsistências nos confrontos futuros.

A regra 4 é aplicada exclusivamente na fase de grupos. Para cada partida concluída, o sistema atualiza a pontuação das seleções envolvidas segundo os critérios estabelecidos:

- Vitória: 3 pontos.
- Empate: 1 ponto para cada seleção.
- Derrota: 0 pontos.

Além da pontuação, são armazenadas estatísticas auxiliares, como gols marcados, gols sofridos e saldo de gols, utilizadas como critérios de desempate durante a classificação dos grupos.

Já a regra 5 é aplicada nas fases eliminatórias. Nessas etapas não é permitido que uma partida termine sem vencedor. Caso o placar termine empatado, o sistema exige a indicação da seleção vencedora nos pênaltis antes de concluir o registro do resultado. O vencedor é então automaticamente encaminhado para o arquivo de classificados da próxima fase.

No tocante à persistência de dados, optou-se pela utilização de arquivos JSON para armazenamento das informações referentes às partidas, resultados e seleções classificadas. Cada

fase da competição possui arquivos próprios para registrar seus confrontos e seus classificados, facilitando a organização dos dados e a manutenção do sistema. A manipulação desses arquivos é realizada por meio da biblioteca Jackson, responsável pela serialização e desserialização dos objetos Java.

Do tratamento de exceções O módulo de partidas, fases e resultados utiliza tratamento de exceções para impedir a inserção de informações inválidas e preservar a integridade dos dados da competição.

Durante o cadastro de partidas, são verificadas situações como a seleção da mesma equipe para ambos os lados do confronto ou a tentativa de registrar seleções inexistentes na fase atual. Nesses casos, é lançada uma exceção do tipo `IllegalArgumentException` contendo uma mensagem explicativa ao usuário.

No registro de resultados, também são realizadas validações para impedir valores negativos de gols, partidas inexistentes ou tentativas de registrar resultados em confrontos que já possuam placar definido. Caso alguma dessas situações seja identificada, uma exceção apropriada é lançada e o registro não é realizado.

Adicionalmente, operações envolvendo leitura e escrita dos arquivos JSON são protegidas por blocos try-catch, permitindo o tratamento de possíveis falhas de entrada e saída (`IOException`). Dessa forma, erros relacionados à persistência dos dados são capturados adequadamente, evitando o encerramento inesperado da aplicação e possibilitando a exibição de mensagens informativas ao usuário.

2.3.4 Estádio e Arbitragem

Foram exigidas três regras de negócio específicas para o gerenciamento de estádios e árbitros da competição:

1. Um estádio não pode sediar duas partidas no mesmo horário.
2. Cada partida deve ter ao menos um árbitro principal.
3. Árbitros não podem atuar em partidas de seleções de sua nacionalidade.

A regra 1 é garantida durante o agendamento das partidas. O sistema realiza uma verificação de disponibilidade do objeto `Estadio` para a data e hora solicitadas. Caso o horário já esteja reservado para outro confronto, é lançada uma `IllegalArgumentException`, bloqueando o agendamento conflitante.

A regra 2 é validada no momento da instanciación da partida. O construtor exige a passagem de um objeto `Arbitro` válido. Se o atributo correspondente ao árbitro principal for nulo (`null`), o sistema interrompe a criação e alerta sobre a falta de equipe de arbitragem.

A regra 3 é assegurada pelo método `validarNacionalidadePartida()` implementado na classe `Arbitro`. Antes de vincular um árbitro a um jogo, o método compara o atributo `nacionalidade` do árbitro com a nacionalidade das duas seleções envolvidas. Em caso de equivalência, uma `IllegalArgumentException` é disparada para impedir conflitos de interesse.

No tocante à persistência de dados, optou-se pela utilização do formato textual JSON. Foi definido que os arquivos gerados fossem salvos no diretório padrão do projeto (`src/main/resources`).

Para a serialização e desserialização, optou-se pela utilização da biblioteca externa Jackson. Através do padrão de projeto DAO, materializado em `GerenciadorEstadioJSON` e `GerenciadorArbitroJSON`, os dados inseridos pelos usuários são persistidos usando a classe `ObjectMapper`. Para a leitura, utilizou-se o `TypeReference` para contornar o apagamento de tipo e garantir a correta reconstrução das listas genéricas (`List<Estadio>`, `List<Arbitro>`).

Por fim, o tratamento de exceções atua como camada de robustez da interface gráfica.

- **NumberFormatException:** Adicionado nos campos de interface gráfica, como o de capacidade do estádio e o de experiência do árbitro, evitando a quebra do software ao receber strings. Caso ocorra essa exceção, é apresentada uma mensagem amigável via `JOptionPane`.
- **IOException:** Aplicado nas classes gerenciadoras a fim de contornar erros de leitura e gravação pelo sistema operacional. Ao capturar o erro, o sistema retorna listas vazias, permitindo que a aplicação inicie de forma estável mesmo sem um arquivo pré-existente.

2.3.5 Ingressos e Público

O módulo de Ingressos e Público foi desenvolvido para gerenciar todo o processo de comercialização de ingressos, controle de público e acompanhamento da arrecadação financeira das partidas da competição. Sua implementação foi baseada na separação das responsabilidades entre as classes do módulo, permitindo que cada componente desempenhasse uma função específica, tornando o sistema mais organizado, reutilizável e de fácil manutenção.

A lógica de negócio está concentrada principalmente na classe `GerenciaIngressos`, responsável pelo processamento das compras e pela aplicação das regras definidas para o módulo. Sempre que uma solicitação de compra é realizada, o sistema verifica inicialmente se a quantidade de ingressos informada é maior que zero e se respeita o limite máximo permitido por compra. Em seguida, é validado o valor informado, impedindo valores negativos e permitindo valores iguais a zero para categorias promocionais, como ingressos VIP gratuitos.

Após as validações iniciais, o sistema verifica se a quantidade solicitada não ultrapassa a capacidade máxima do estádio. Caso todas as condições sejam satisfeitas, a compra é concluída com sucesso, atualizando automaticamente a quantidade de ingressos vendidos e o valor arrecadado para a partida correspondente. Caso alguma regra seja violada, uma exceção é lançada e a operação é cancelada, preservando a consistência das informações armazenadas.

As informações referentes às vendas são mantidas pela classe `DadosIngressos`, que utiliza estruturas do tipo `HashMap` para armazenar a quantidade de ingressos vendidos e os valores arrecadados de cada partida. A utilização dessa estrutura permite consultas e atualizações eficientes, utilizando a identificação da partida como chave de acesso.

Com o objetivo de garantir a persistência das informações entre diferentes execuções do sistema, foi implementado um mecanismo de armazenamento utilizando o arquivo `eventos.json`. Após cada compra realizada, os dados são automaticamente gravados nesse arquivo por meio do método `salvarEventosJson()`, permitindo que as vendas realizadas não sejam perdidas quando a aplicação é encerrada. Durante a inicialização do sistema, a classe `DadosIngressos`

realiza automaticamente a leitura desse arquivo através do método `carregarEventosJson()`, restaurando os dados financeiros anteriormente registrados.

Além do processamento das vendas, a classe `GerenciaIngressos` disponibiliza métodos responsáveis pelo cálculo dos principais indicadores estatísticos do módulo, como arrecadação total da competição, média de público e evento de maior arrecadação. Essas informações são utilizadas diretamente pelo relatório financeiro apresentado ao usuário.

Também foi implementada a funcionalidade *Nova Temporada*, responsável por reinicializar as estatísticas de vendas, público e arrecadação, além de limpar o conteúdo do arquivo `eventos.json`. Essa operação preserva as partidas cadastradas no sistema, reiniciando apenas os dados estatísticos relacionados às vendas de ingressos.

A implementação do módulo seguiu os princípios da Programação Orientada a Objetos, priorizando o encapsulamento, a separação de responsabilidades entre as classes e a centralização das regras de negócio em uma única camada. Essa abordagem facilita futuras manutenções, reduz o acoplamento entre os componentes do sistema e permite a evolução do módulo sem comprometer as demais funcionalidades da aplicação.

2.4 Integração e navegabilidade

2.4.1 Administração e Gestão

Os métodos de administração, anteriormente implementados, foram inseridos em suas respectivas telas. Além disso, de acordo com o princípio do RBAC e da coleção de permissões, cada tela é habilitada de acordo com as ações que o usuário logado pode realizar. Isso, inclusive, é feito pelo padrão de design Injeção de Dependência, que passa as informações da sessão para cada tela em que sejam necessárias, uma boa alternativa ao Singleton. Vale ressaltar que cada relatório está em sua respectiva área. A exemplo, caso queiramos obter o relatório de partidas, devemos ir à aba partidas. Isso foi feito para facilitar a administração de permissões, visto que retira a necessidade de habilitar e desabilitar abas desconexas.

2.4.2 Jogadores e Seleções

No tocante ao módulo de Jogadores e Seleção, a integração da lógica sistemática com a Graphical User Interface (GUI) foi aplicada através da inserção direta dos algoritmos de escrita e leitura de registros nos métodos para eventos (`java.awt.event`) de botão pressionado gerados pela ferramenta de Design do Ambiente de Desenvolvimento Integrado (IDE) NetBeans. A fim de aumentar a legibilidade do código, os algoritmos foram micro-modularizados em métodos internos menores, a exemplo `aplicaFiltro()` e `atualizaTabela()`.

Em especial, as telas `CadastroJogador` e `CadastroSelecao` tiveram seus construtores sobrecarregados para que atuassem distintamente nos contextos de cadastro e de edição das classes `Jogador` e `Selecao`, respectivamente; reaproveitando, desse modo, o código de ambos os displays.

Por fim, fez-se necessário sobrescrever os métodos `isCellEditable()`, `getColumnClass()` (ambos nativos da classe `DefaultTableModel`) e `valueChanged()` (nativo da classe `ListSelectionEvent`) para fornecer uma interação fluída do usuário com as tabelas; as quais foram criadas a partir da classe `JTable`, nativa da biblioteca Abstract Window Toolkit (AWT).

2.4.3 Estádios e Arbitragem

No módulo de Estádios e Arbitragem, a integração entre a interface gráfica e as regras de negócio foi consolidada por meio do mapeamento de eventos, conectando os formulários de cadastro e os botões de ação diretamente à camada de persistência. A navegabilidade foi estruturada para garantir um fluxo de uso consistente; as transições para telas específicas — como a visualização das partidas designadas a um árbitro — ocorrem de forma fluida através da instanciamento de novos contêineres (JFrame), com o devido gerenciamento de fechamento (DISPOSE_ON_CLOSE) para evitar o encerramento abrupto do sistema principal.

Para a exibição e integração de dados, o módulo faz uso intenso da manipulação de tabelas (JTable) alocadas em painéis de rolagem (JScrollPane). A extração e atualização das informações na interface ocorrem de maneira dinâmica operando sobre o DefaultTableModel, refletindo imediatamente as alterações do banco de dados na GUI.

Por fim, a comunicação entre as telas e o controle de acesso foram assegurados mantendo o padrão de passagem do estado da sessão (usuário logado) através da sobrecarga dos construtores das interfaces visuais. Aliada ao uso de herança no modelo lógico de usuários, essa arquitetura garante que a injeção de dependência restrinja a execução de funcionalidades sensíveis e a exibição de determinados componentes apenas a perfis com a devida autorização no sistema.

2.4.4 Partidas

Em relação ao módulo de Partidas e Resultados, a integração entre a interface gráfica Swing e a camada de negócio foi realizada através dos eventos dos componentes visuais, conectando as telas de criação de partidas, registro de resultados e consulta às classes responsáveis pelo gerenciamento das fases da competição.

A criação de partidas utiliza componentes como JComboBox e JTextField para selecionar fase, seleções, estádio, árbitro, data e horário. Os dados são validados antes da persistência, garantindo o cumprimento das regras de negócio, como a impossibilidade de uma seleção enfrentar a si mesma ou de um árbitro apitar partidas envolvendo seu país de origem.

No registro de resultados, a interface permite localizar partidas previamente cadastradas e atualizar seus placares. Para fases eliminatórias, também é possível definir o vencedor nos pênaltis, sendo as informações repassadas às classes especializadas de cada fase para atualização dos arquivos de persistência.

Já a consulta de partidas foi implementada por meio da seleção da fase desejada, permitindo a recuperação e exibição dinâmica das partidas armazenadas. Essa abordagem mantém o desacoplamento entre a camada visual e a lógica da aplicação, concentrando as regras de negócio nas classes de domínio e utilizando a interface apenas como mecanismo de interação com o usuário.

2.4.5 Ingressos e Público

O módulo de Ingressos e Público foi desenvolvido de forma integrada aos demais módulos do sistema, permitindo o compartilhamento automático de informações entre as diferentes funcionalidades da aplicação. Essa integração elimina a duplicidade de dados e garante maior consistência entre as informações apresentadas ao usuário.

A principal integração ocorre com o módulo de Partidas. Durante a inicialização da tela de compra de ingressos, o sistema consulta automaticamente as partidas cadastradas através da classe **FaseGrupos**. Dessa forma, qualquer partida criada pelo usuário passa imediatamente a estar disponível para comercialização de ingressos, sem necessidade de cadastro adicional ou sincronização manual entre os módulos.

Além disso, o sistema verifica automaticamente o estado de cada partida. Apenas confrontos que ainda não possuem resultado registrado são apresentados na tela de venda de ingressos. Quando o resultado de uma partida é informado pelo módulo de Partidas, ela deixa automaticamente de aparecer entre as opções disponíveis para compra, impedindo a venda de ingressos para jogos já encerrados e garantindo a consistência das regras de negócio.

A integração também ocorre com o módulo responsável pelos relatórios financeiros. Após cada compra realizada, os métodos da classe **GerenciaIngressos** atualizam imediatamente a quantidade de ingressos vendidos, a arrecadação da partida e os indicadores financeiros da competição. Essas informações são persistidas no arquivo `eventos.json` e disponibilizadas automaticamente para a tela de relatório financeiro, que apresenta dados sempre atualizados ao usuário.

A navegação entre as funcionalidades foi organizada por meio do **MenuPrincipal**, permitindo acesso às telas de compra de ingressos, controle de público e relatório financeiro. Essa organização proporciona uma interface intuitiva, reduzindo a quantidade de operações necessárias para acessar cada funcionalidade e facilitando a utilização do sistema.

Durante a utilização do módulo, o fluxo de execução ocorre de forma transparente ao usuário. Inicialmente, as partidas são cadastradas pelo módulo de Partidas. Em seguida, essas partidas tornam-se automaticamente disponíveis para venda de ingressos. Após a realização das compras, os dados financeiros são atualizados, persistidos em arquivo JSON e imediatamente refletidos nos relatórios financeiros da aplicação. Essa integração entre os módulos permite que todas as informações permaneçam sincronizadas durante toda a execução do sistema.

Com essa arquitetura, o módulo de Ingressos e Público deixa de funcionar de forma isolada e passa a atuar em conjunto com os demais componentes da aplicação, promovendo reutilização de informações, redução de inconsistências e maior automatização do gerenciamento da Copa do Mundo.

3 Resultados e Discussão

3.1 Tecnologia Utilizadas

Além dos recursos utilizados na seção 2.1, foram empregadas as Interfaces de Programação de Aplicações (API):

- **Jackson:** Biblioteca de métodos para manipulação de dados JSON em Java. Utilizada na leitura e escrita dos arquivos de persistência.
- **JasperReports:** Framework open source voltado para fabricação de documentos na plataforma Java, destacando-se seu aplicativo Jaspersoft Studio. Utilizado para ela-

oração dos relatórios em Portable Document Format (PDF).

3.2 Demonstração da Solução

As imagens a seguir apresentam a versão final da aplicação gerada.

3.2.1 Módulo Inicial



Figura 2: Tela Inicial

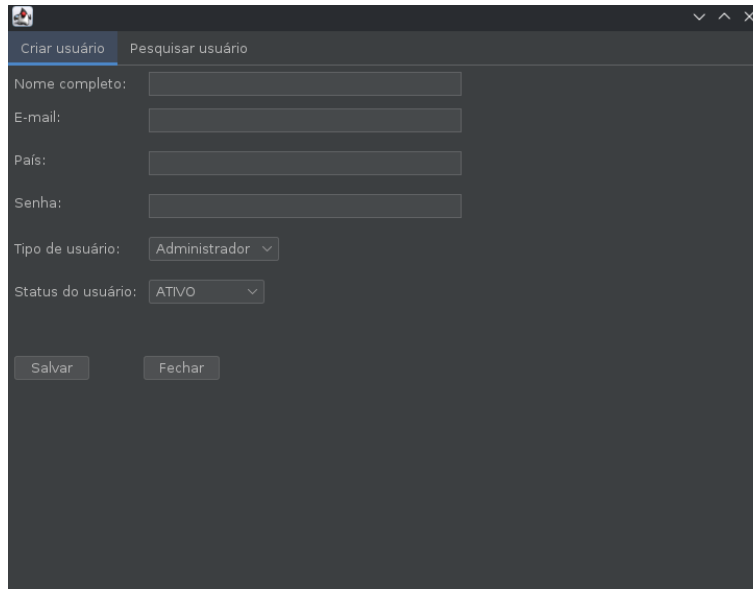
A Tela Inicial é apenas uma apresentação do programa a ser explorado nas telas seguintes.



Figura 3: Tela Login

A Tela Login permite ao usuário acessar sua área de trabalho a partir de um login (CPF) e de uma senha. A existência desses dados é verificada no Banco de Dados para liberar o acesso.

3.2.2 Módulo Administrador

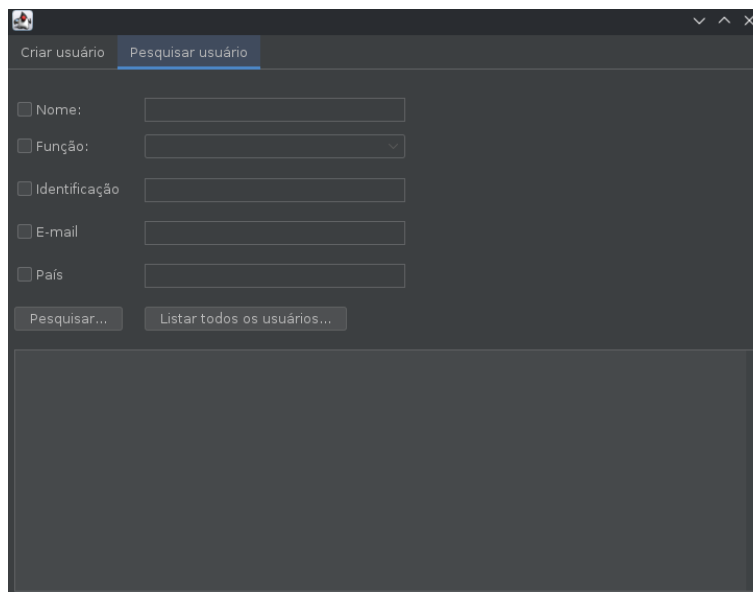


The screenshot shows a window titled 'Criar usuário' with a sub-tab 'Pesquisar usuário'. The form contains the following fields and controls:

- Nome completo:
- E-mail:
- País:
- Senha:
- Tipo de usuário:
- Status do usuário:
- Buttons: Salvar, Fechar

Figura 4: Tela de criação de usuário

A Tela para a Criação de Usuários proporciona a possibilidade de cadastro de novos administradores, organizadores, operadores e árbitros como usuários do sistema.



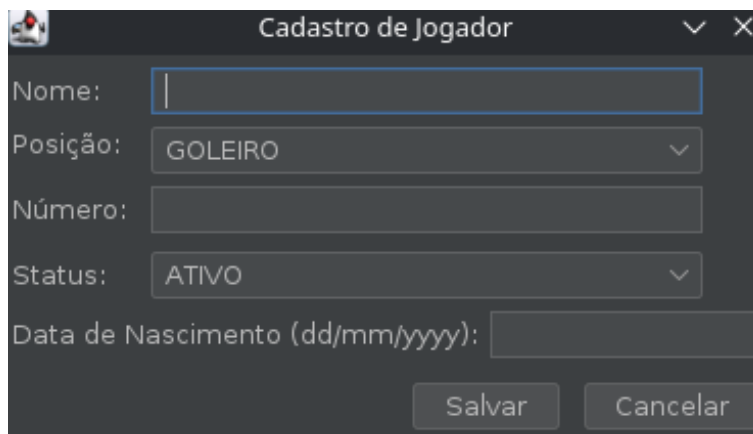
The screenshot shows a window titled 'Pesquisar usuário' with a sub-tab 'Criar usuário'. The form contains the following fields and controls:

- Nome:
- Função:
- Identificação:
- E-mail:
- País:
- Buttons: Pesquisar..., Listar todos os usuários...

Figura 5: Tela de pesquisa de usuário

Essa tela permite o usuário pesquisar os diferentes usuários do sistema.

3.2.3 Módulo Seleções/Jogadores



Cadastro de Jogador

Nome:

Posição: GOLEIRO

Número:

Status: ATIVO

Data de Nascimento (dd/mm/yyyy):

Salvar Cancelar

Figura 6: Tela de Cadastro de Jogador

A Tela Cadastro de Jogador permite o registro de um Jogador. Nela é obrigatório o preenchimento dos atributos Nome, Posição, Número, Status e Data de Nascimento.

Cadastro de Seleção

País:

Grupo:

Técnico:

Selecione 18 a 26 jogadores.

0 jogadores selecionados.

Nome	Número	Posição
Manuel Neuer	1	GOLEIRO
Kai Havertz	19	ATACANTE
Declan Rice	23	MEIO-CAMPISTA
Elliot Anderson	20	MEIO-CAMPISTA
Harry Kane	9	ATACANTE
Marcus Rashford	11	ATACANTE
Anthony Gordon	18	ATACANTE
Bart Verbruggen	1	GOLEIRO
Mark Flekken	3	GOLEIRO
Virgil Van Dijk	4	DEFENSOR
Akam Hashim	5	DEFENSOR
Youssef Aryn	15	GOLEIRO
Ali Abdi	2	DEFENSOR
Montassar Talbi	3	DEFENSOR
Ismalia Sarr	17	ATACANTE
Iliman Ndiaye	19	ATACANTE
Cherif Ndiaye	21	ATACANTE
Kar Sams	12	MEIO-CAMPISTA

Salvar

Cancelar

Figura 7: Tela de Cadastro de Seleção

A Tela Cadastro de Seleção permite a inscrição de uma Seleção. Nela é compulsório que os dados de País, Grupo e Técnico sejam informados, além da escolha de 18 a 26 jogadores para serem agregados pela Seleção.

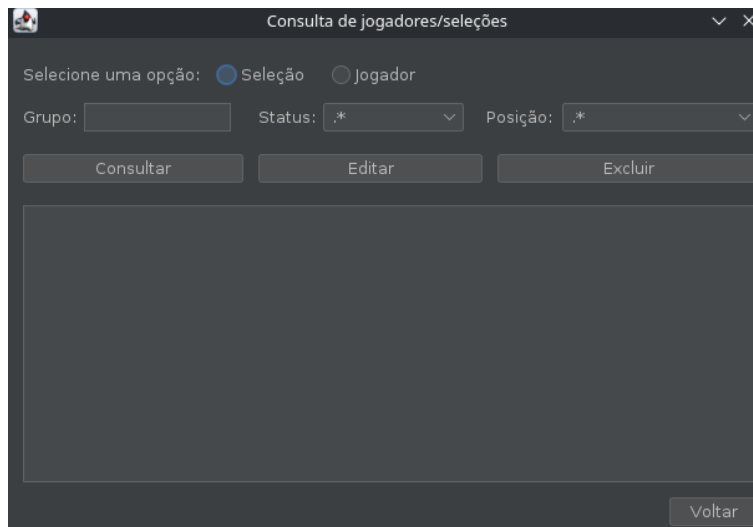


Figura 8: Tela de Consulta de Jogador/Seleção

A Tela de Consulta de Jogador ou Seleção permite ao usuário consultar os jogadores ou seleções armazenados nos arquivos de persistência jogadores.jsonl e selecoes.jsonl, respectivamente. Na tela, é disponibilizada a opção de exibição para escolha do tipo a ser consultado (Jogador ou Seleção), juntamente filtros de "Status" e "Posição" para a listagem de Jogadores e o filtro "Grupo" para a listagem de Seleções

3.2.4 Módulo Partidas

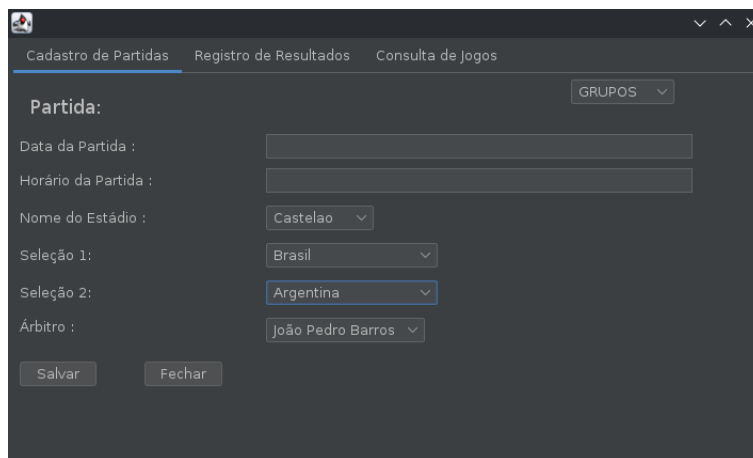


Figura 9: Tela de criação de partida

A Tela de Criação de Partidas permite o cadastro de partidas da competição. Nela são informadas a fase, as seleções participantes, o estádio, o árbitro, a data e o horário.

Cadastro de Partidas Registro de Resultados Consulta de Jogos

Digite o Número da Partida: Fase: GRUPOS

Buscar

Gols da Seleção 1: Gols da Seleção 2:

(Caso haja um empate depois da Fase de Grupos) Qual seleção ganhou nos pênaltis? :

Item 1

Salvar

Figura 10: Tela de registro de partida

A Tela de Registro de Resultados permite registrar o placar das partidas já cadastradas. Em fases eliminatórias, também é possível informar o vencedor nos pênaltis.

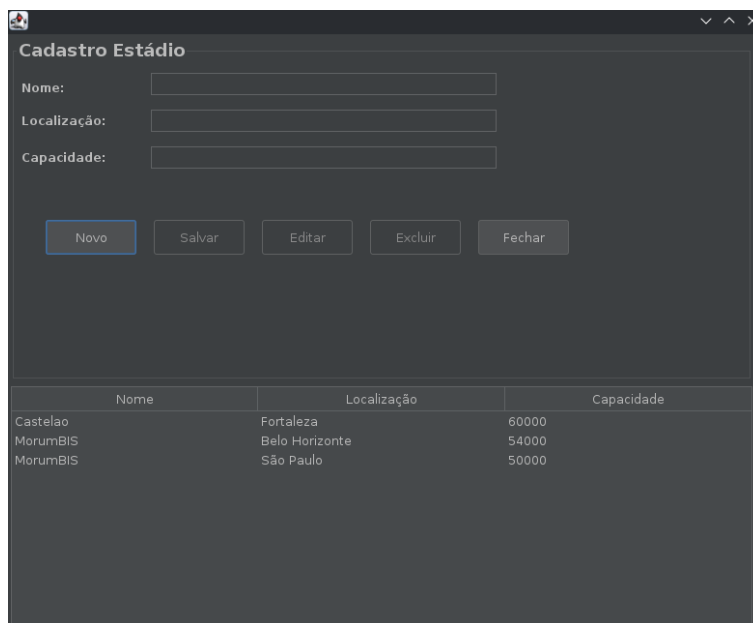
Cadastro de Partidas Registro de Resultados Consulta de Jogos

Mostrar GRUPOS

Figura 11: Tela de consulta de partida

A Tela de Consulta de Partidas permite visualizar as partidas cadastradas em cada fase da competição. São exibidas informações como seleções participantes, placar e vencedor.

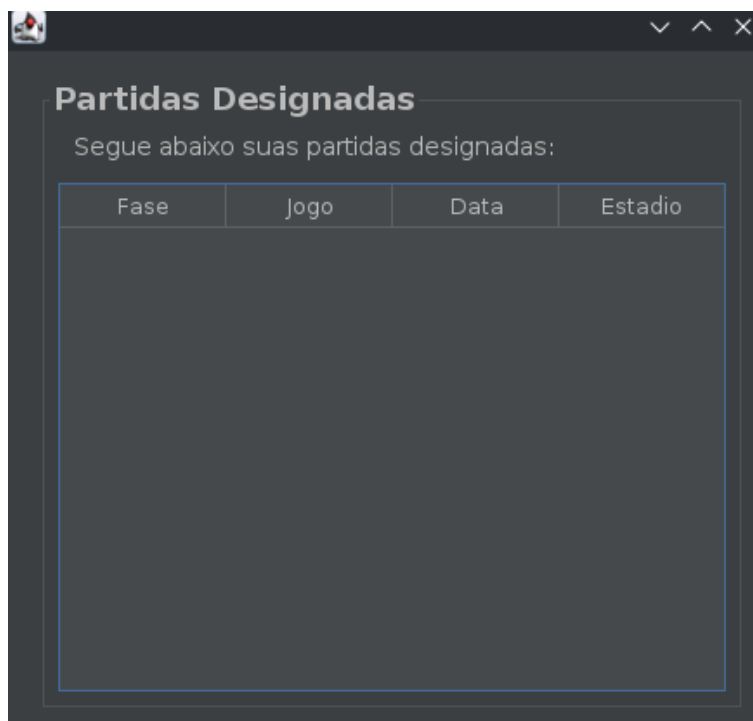
3.2.5 Módulo Estádios/Arbitragem



Nome	Localização	Capacidade
Castelao	Fortaleza	60000
Morumbi	Belo Horizonte	54000
Morumbi	São Paulo	50000

Figura 12: Tela de cadastro de estádio

A Tela de Cadastro e Consulta de Estádios permite registrar e visualizar os estádios utilizados na competição. São informados nome, localização e capacidade do estádio.

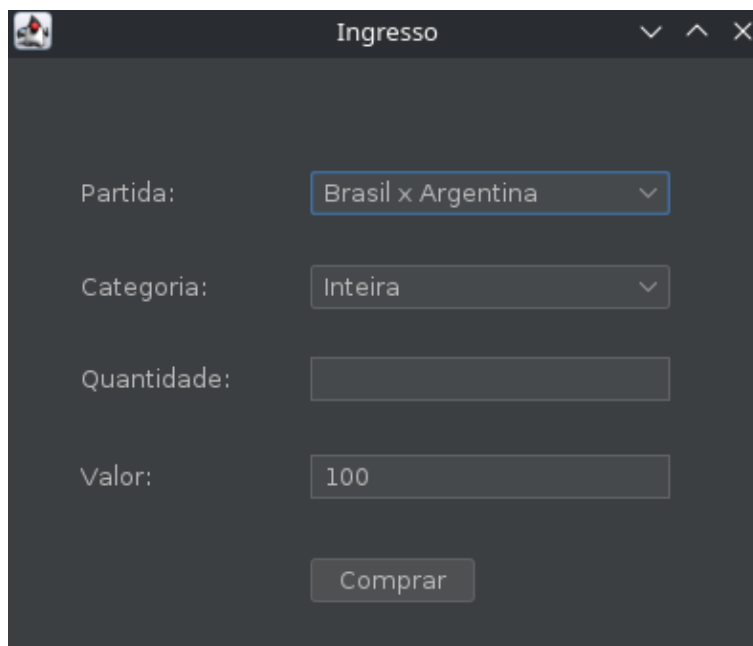


Fase	Jogo	Data	Estadio
------	------	------	---------

Figura 13: Tela de designação de árbitro

A Tela de Designação de Árbitros permite que os árbitros consultem as partidas para as quais foram designados. São exibidas informações como seleções, estádio, data e horário.

3.2.6 Módulo Público/Ingressos



Ingresso

Partida: Brasil x Argentina

Categoria: Inteira

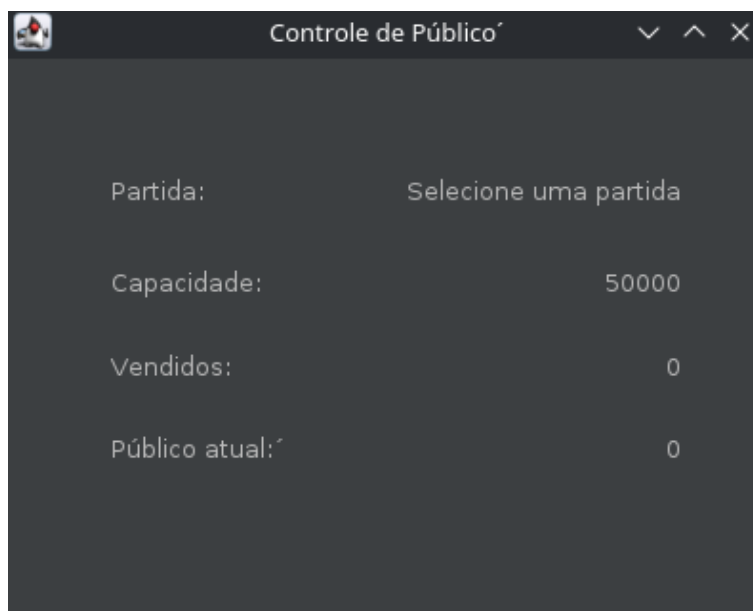
Quantidade:

Valor: 100

Comprar

Figura 14: Tela de compra de ingressos

A Tela de Compra de Ingressos permite que os usuários adquiram ingressos para as partidas da competição.



Controle de Público

Partida: Selecione uma partida

Capacidade: 50000

Vendidos: 0

Público atual: 0

Figura 15: Tela de controle de público

A Tela de Controle de Público permite acompanhar a ocupação dos estádios durante a competição. São exibidos dados de capacidade, ingressos vendidos e público atual.

Partida:	Selecione uma partida
Ingressos vendidos:	0
Total arrecadado:	R\$ 0,00
Média de Publico:	0 pessoas
Maior Arrecadação:	
TOTAL GERAL:	R\$ 0,00

Nova Temporada

Figura 16: Tela de relatórios financeiros

A Tela de Relatório Financeiro permite consultar os valores arrecadados com a venda de ingressos.

3.3 Técnicas de programação Aplicadas

Durante o desenvolvimento do sistema, foram aplicadas técnicas de orientação a objetos e padrões de projeto com o objetivo de aumentar a modularidade, reutilização e manutenção do código. Entre elas, destacam-se o uso de Dependency Injection e do padrão Template Method.

O princípio de Dependency Injection foi utilizado principalmente no módulo de administração de usuários. Em vez de criar internamente os objetos responsáveis pela persistência e gerenciamento dos usuários, essas dependências são fornecidas às classes e interfaces que necessitam delas. Essa abordagem reduz o acoplamento entre os componentes do sistema, facilita testes e permite a reutilização da mesma lógica de gerenciamento em diferentes telas da aplicação.

Já o padrão Template Method foi empregado nas hierarquias de permissões e fases da competição. Na estrutura de permissões, a classe abstrata define o comportamento geral para controle de acesso, enquanto as subclasses implementam permissões específicas para cada perfil de usuário. De forma semelhante, a classe abstrata 'Fase' estabelece as operações comuns às etapas da competição, como criação de partidas, registro de resultados e geração de classificados, deixando para as subclasses (FaseGrupos, OitavasFinal, QuartasFinal, entre outras) a implementação dos comportamentos particulares de cada fase.

A utilização dessas técnicas contribuiu para a organização da arquitetura do sistema, promovendo maior extensibilidade e facilitando a implementação de novas funcionalidades

sem a necessidade de alterar componentes já consolidados.

Nos demais módulos do sistema, embora tenham sido amplamente utilizados conceitos de orientação a objetos, como encapsulamento, herança, composição e polimorfismo, não houve a adoção explícita de um padrão de projeto específico, uma vez que as soluções implementadas já atendiam adequadamente aos requisitos funcionais propostos.

3.4 Desafios Encontrados e Soluções Adotadas

Durante o desenvolvimento deste projeto, foram encontrados diversos desafios. Um deles envolveu conflitos de controle de versão, nos quais o sistema impedia a execução do comando pull para obter as atualizações do repositório no GitHub. Para solucionar esse impasse, foram utilizados comandos de redefinição de estado local (reset) e, em alguns casos, o envio forçado de código (force push). Outro problema recorrente foi a interferência de serviços de armazenamento em nuvem (como o OneDrive), que bloqueava arquivos de cache temporários do Git durante operações críticas. A solução prática foi pausar a sincronização do serviço sempre que fosse necessário enviar ou receber código.

No escopo da interface gráfica, um desafio notável foi o comportamento anômalo na instanciação de uma das telas baseadas em JFrame, que resultava em uma espécie de "tela fantasma" vazia, não renderizando a interface da forma correta. A solução implementada foi reestruturar a chamada visual e forçar o dimensionamento inicial da janela instanciada para que o gerenciador de layout calculasse e exibisse os componentes adequadamente.

3.5 Análise Crítica da Qualidade da Solução

Ao analisar categoricamente o software desenvolvido, nota-se que ele atende a todos os requisitos funcionais (regras de negócio) e não-funcionais exigidos; apresentando satisfatória aceitabilidade, além de suficiente grau de confiança, segurança e eficiência, considerado o escopo de uso do programa. Contudo, a aplicação demonstra baixa manutenibilidade e capacidade de reutilização, dada a ausência de encapsulamento de todos os módulos em interfaces; além de um considerável acoplamento entre as classes voltadas à persistência de dados e as voltadas à interface gráfica.

4 Conclusão e Evolução Futura

A produção do sistema aqui descrito, mostrou-se de grande valia para o aprendizado dos membros no tocante à área de Técnicas de Programação. Proporcionando uma experiência prática das ferramentas discutidas em sala de aula e promovendo um entendimento profundo da importância e da utilidade desse campo de estudo.

Ademais, concluiu-se que a evolução deste trabalho deverá passar, a princípio, pela refatoração do código, seguindo uma convenção de padrões de projeto amplamente utilizados no mercado de trabalho, a exemplo Factory, Builder, Adapter e Command. Tal procedimento busca viabilizar que a criação e estruturação de objetos e classes, além do comportamento das funcionalidades da aplicação sejam de fácil reutilização e manutenção para versões futuras do projeto; além de prover um ambiente favorável à expansão de operacionalidades.

Referências

- [1] Jaspersoft Community. Documentação jaspersoftstudio. <https://community.jaspersoft.com/documentation/>. Acessado em jun. 2026.
- [2] Oracle. Documentação java 25. <https://docs.oracle.com/en/java/javase/25/>. Acessado em jun. 2026.
- [3] Sadiul Hakim. Jackson tutorial. https://dev.to/sadiul_hakim/jackson-tutorial-comprehensive-guide-with-examples-2gdj. Acessado em mai. 2026.